

CivicRAG Phase 2 Update

Bilingual Bangla-English Retrieval-Augmented Generation system for Bangladeshi government-service question answering. Current domain: Birth and Death Registration.

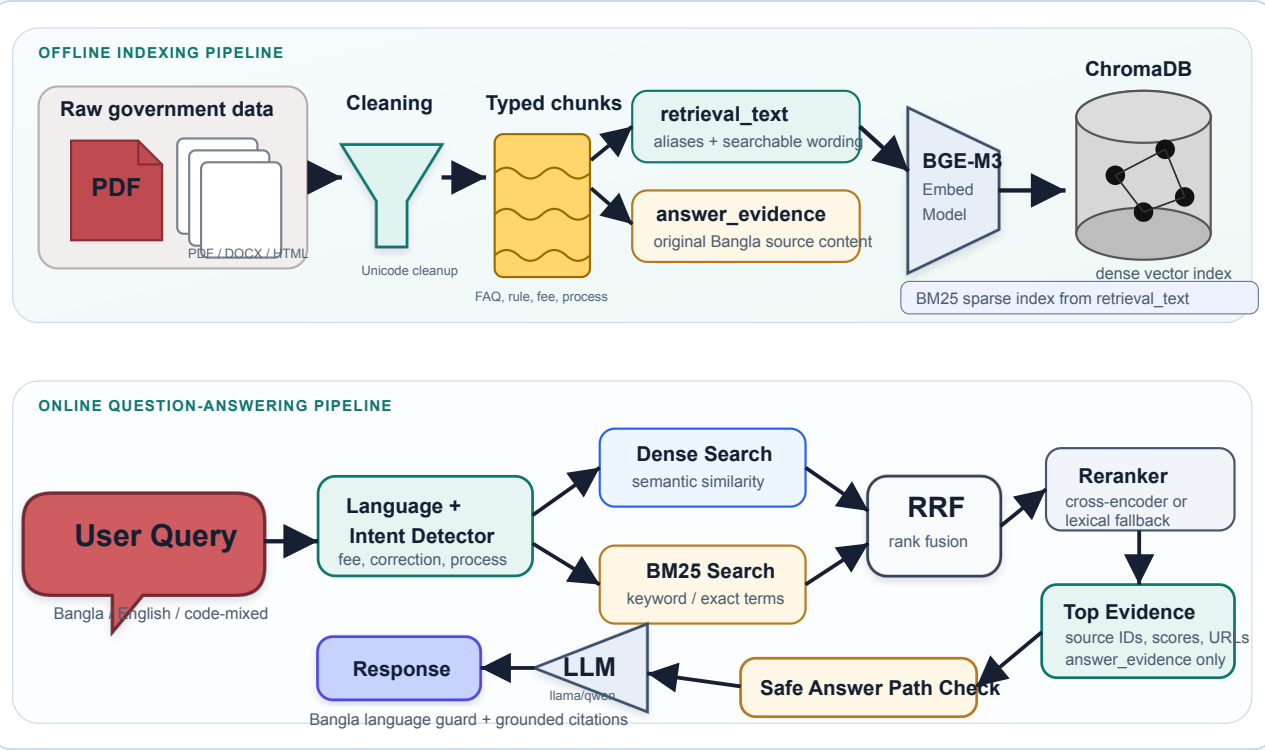
SELECTED DESIGN CivicRAG: hybrid retrieval + grounded local LLM generation	EMBEDDING BAAI/bge-m3	LOCAL LLMS llama3.2, llama3, qwen2.5:7b
--	---	---

Design Process or Methodology Overview

The objective of this phase is to design and verify a bilingual Bangla-English Retrieval-Augmented Generation (RAG) system for Bangladeshi government-service question answering. The current working domain is birth and death registration. The design is inspired by the NextRAG framework proposed for bilingual financial question answering, but adapted to civic-service documents, Bangla-heavy data, and local LLM deployment.

The main design constraints are factual correctness, evidence grounding, Bangla query handling, explainability, and future multi-domain extensibility. Therefore, the system is designed as a modular RAG pipeline instead of a single monolithic chatbot. The pipeline separates data ingestion, cleaning, chunking, embedding, retrieval, reranking, answer generation, and evaluation.

Figure 1. Proposed CivicRAG architecture for birth and death registration QA



The architecture separates offline indexing from online answering. Aliases are used only in `retrieval_text`; the generator receives original source evidence to reduce unsupported claims.

The design process followed four steps. First, raw civic-service documents were collected from government-service sources. Second, the data was cleaned and converted into retrieval-friendly Markdown/JSON formats. Third, multiple retrieval alternatives were implemented and evaluated: dense-only retrieval, BM25-only retrieval, hybrid RRF retrieval, and reranked hybrid retrieval. Fourth, local LLM backends were tested using Ollama models to avoid API dependency and preserve reproducibility.

$$\text{RRF}(d) = \sum_{r \in R} \frac{w_r}{k + \text{rank}_r(d)}$$

Here, d is a document chunk, R is the set of retrievers, w_r is the retriever weight, k is the RRF constant, and $\text{rank}_r(d)$ is the rank of chunk d under retriever r .

Preliminary Design or Design (Model) Specification

Several alternative designs were considered for the birth and death registration RAG system.

Alternative RAG design choices considered in Phase 2.

Design	Description	Reason for Evaluation
Dense-only RAG	Uses BGE-M3 embeddings and ChromaDB vector similarity search.	Strong for Bangla paraphrases and semantic matching.
BM25-only RAG	Uses lexical keyword matching with BM25.	Useful for exact terms such as fees, rules, form names, and service names.
Hybrid RAG	Combines dense retrieval and BM25 using RRF.	More robust for future multi-domain settings such as passport, BRTA, and certificates.
CivicRAG	Hybrid retrieval, metadata, source URLs, language guard, safe-answer paths, and local LLM generation.	Proposed system optimized for factual government-service QA.

The selected implementation currently uses `BAAI/bge-m3` as the multilingual embedding model, ChromaDB as the vector index, BM25 as the sparse retriever, RRF for result fusion, and local Ollama-hosted LLMs for generation. The tested local LLMs are `llama3.2`, `llama3`, and `qwen2.5:7b`.

For verification, a 58-question Bangla paraphrase evaluation set was prepared from 19 grouped civic-service problems. The retrieval variants were evaluated using Hit@k, MRR, nDCG@5, and Context Precision.

Retrieval variant comparison on the current birth/death registration evaluation set.

Method	Hit@1	Hit@5	MRR	nDCG@5
BM25-only	0.5345	0.7586	0.6132	0.6112
Dense-only	0.7931	0.9828	0.8707	0.8693
Hybrid RRF	0.7414	0.9483	0.8193	0.8139

The result shows that dense-only retrieval currently performs best in the single-domain Bangla paraphrase setting. However, hybrid retrieval remains part of the proposed system because future domains may contain overlapping service language where lexical/domain-specific matching can reduce cross-domain confusion.

Generation context top-k coverage.

k	Expected Source in Context	Context Precision@k
1	0.7414	0.7414
3	0.8793	0.3621
5	0.9483	0.2448
8	0.9828	0.1616

The current selected value is k=3 for generation because it is conservative and reduces noise. However, k=5 is a candidate for future testing because it gives higher expected-source coverage.

Data Collection -(If Applicable)

The current dataset focuses on Bangladeshi birth and death registration services. The data was collected from government-service documents and web pages related to birth registration, death registration, correction, fees, rules, frequently asked questions, and BDRIS service procedures.

The collected data includes four types of raw sources:

- Bijoy-encoded PDF documents.
- Converted DOCX files.
- Unicode HTML government web pages.
- Scanned PDF/OCR-based documents.

The raw sources were converted into cleaned Markdown and JSON formats before indexing. JSON was used mainly for FAQ-style question-answer pairs, while Markdown was used for heading-based procedural, legal, and informational documents.

Data Cleaning

The cleaning process focused on removing noise that can harm retrieval quality. Government web pages often contain menus, accessibility controls, repeated navigation text, news blocks, footer links, and unrelated service lists. These were removed where possible.

- Removing duplicate web-navigation blocks and repeated footer sections.
- Converting Bijoy/non-Unicode text into Unicode Bangla where possible.
- Separating FAQ entries into question-answer units.
- Keeping official service links as metadata instead of mixing them randomly into answer text.
- Removing empty headings, broken OCR fragments, and irrelevant page boilerplate.
- Preserving legal/procedural wording where factual precision is important.

Data Transformation

The cleaned documents were transformed into retrieval-friendly chunks. Instead of using only fixed-size chunking, the system uses document-type-aware chunking. This is important because civic-service documents contain different structures.

- FAQ data is chunked as one question-answer pair per chunk.
- Heading-based Markdown documents are chunked by section headings and subheadings.
- Table-like fee data is converted into row-level chunks.
- Legal/rules documents are chunked by rule, section, or meaningful paragraph group.
- Long chunks are limited using minimum and maximum character thresholds.

Each chunk contains two text versions. The first is `retrieval_text`, which may include aliases and query-matching terms. The second is `answer_evidence`, which stores the original Bangla source content for grounding the LLM answer. This separation reduces hallucination risk because aliases help retrieval, while the LLM only receives source-grounded evidence.

Data Integration

All cleaned sources are integrated under one domain folder: `domains/birth_death_registration/`. The domain folder contains raw data, cleaned/interim chunks, ChromaDB vector storage, configuration files, and evaluation datasets. This structure supports future domains without creating separate repositories.

Data Reduction

Data reduction was performed through chunk selection and metadata filtering rather than numerical dimensionality reduction. The goal was to reduce noisy text while preserving evidence needed for accurate answers.

- Removing repeated government website menus and footer text.
- Skipping duplicate FAQ content where the same answer appeared in multiple sources.
- Keeping only meaningful service-related sections.
- Chunking large documents into focused evidence units.
- Using top-k retrieval and reranking to select only the most relevant chunks for generation.

Summary of Preprocessed Data

The final preprocessed dataset contains Bangla-heavy birth/death registration chunks with metadata such as document type, section title, source ID, category, and official URL. The indexed knowledge base includes FAQ chunks, legal-rule chunks, fee-table chunks, correction-process chunks, application-process chunks, and general guidance chunks.

The current evaluation dataset contains 58 Bangla paraphrase questions across 19 grouped civic-service problems. These questions test whether the system can retrieve the same expected evidence even when the user asks the problem in different wording.

Implementation of Selected Design

The selected design, named CivicRAG, was implemented as a local RAG system. The implementation uses modular Python components for chunking, embedding, indexing, retrieval, reranking, generation, and evaluation.

The embedding model is `BAAI/bge-m3`, selected because it supports multilingual retrieval and is suitable for Bangla-English and cross-lingual settings. The vector database is ChromaDB. Sparse retrieval is implemented using BM25. Dense and sparse retrieval results are combined using RRF. The current RRF weights are 0.55 for dense retrieval and 0.45 for BM25. The default local LLM is `llama3.2`, with `llama3` and `qwen2.5:7b` used for comparison.

The current generation temperature is set to 0.05 to reduce randomness and support factual civic-service answers. This is appropriate because the system prioritizes correctness over creativity.

For high-confidence civic intents such as fees, correction, lost certificate, and application process, the system uses controlled answer paths. If no safe path is available, the system uses the local LLM with retrieved Bangla evidence. The answer includes source IDs, retrieval scores, and official/source links for explainability.

The implementation was verified using both retrieval-side and generation-side experiments. Retrieval-side evaluation used Hit@k, MRR, nDCG@5, and Context Precision. Generation-side evaluation compared local LLM outputs using semantic correctness, token F1, answer relevance, faithfulness proxy, expected-source retrieval, and latency.

The current result supports the Phase 2 objective because multiple alternative designs were implemented and simulated before selecting the current CivicRAG design. Dense-only retrieval is the strongest current baseline, while CivicRAG remains the selected design because it is more explainable, extensible, and suitable for future multi-domain government-service expansion.

References

1. Opin et al., “NextRAG: A Bilingual Retrieval-Augmented Generation Framework for Financial Question-Answering,” ICCIT, IEEE, 2025.
2. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
3. Robertson and Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, 2009.
4. Cormack, Clarke, and Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” SIGIR, 2009.
5. Chen et al., “BGE M3-Embedding,” arXiv:2402.03216, 2024.
6. Dubey et al., “The Llama 3 Herd of Models,” arXiv:2407.21783, 2024.
7. Qwen Team, “Qwen2.5 Technical Report,” 2024.